

Trilha de Node.js

*Módulo 05: Templating e Interface
com Servidor*

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.
2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.
3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.
4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.
5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.
6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.
7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.
8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 5: Server-Side Rendering, Templating e Interface com Servidor

Objetivo:

Capacitar os alunos a renderizar páginas dinâmicas utilizando dados provenientes do MySQL, com foco em organização de layouts e integração eficiente entre front-end e back-end.

5.1 SSR (Server-Side Rendering) no Node.js

Server-Side Rendering (SSR) é uma técnica de renderização de páginas web em que o código HTML é gerado no servidor e enviado ao cliente (navegador). Em contraste com o Client-Side Rendering (CSR), onde a renderização é feita no navegador, o SSR oferece vantagens como:

- **Melhoria no SEO:** Motores de busca conseguem indexar o conteúdo imediatamente, pois ele já está renderizado.
 - **Carregamento mais rápido para o usuário:** O HTML inicial chega ao navegador já processado.
 - **Melhor desempenho em dispositivos de baixa potência:** Reduz a carga de processamento no cliente.
-

Como funciona o SSR?

- **Requisição do cliente:** O navegador do usuário faz uma requisição HTTP ao servidor.
 - **Renderização no servidor:** O servidor processa a requisição, gera o HTML dinâmico e retorna ao cliente.
 - **Entrega ao cliente:** O navegador recebe o HTML completo e o exibe imediatamente.
 - **Hidratação (opcional):** Caso o SSR seja combinado com frameworks de frontend (como React), o JavaScript carrega e torna a página interativa.
-

Aplicabilidade do SSR

- **Sites orientados a conteúdo:** Blogs, portais de notícias e sites de e-commerce.
 - **SEO é crucial:** Sites que dependem de visibilidade em motores de busca.
 - **Dispositivos com hardware limitado:** SSR reduz a carga de renderização no cliente.
 - **Páginas que necessitam de carregamento rápido inicial.**
-

Tecnologias e Ferramentas Usadas com Node.js para SSR

1. Express.js

Express é uma biblioteca minimalista que simplifica a criação de servidores Node.js.

2. Templates Engines

Engines de template geram HTML dinâmico baseado em dados e lógica de renderização. Exemplos populares incluem:

- **Pug (anteriormente Jade)**
- **Handlebars**
- **EJS (Embedded JavaScript)**

3. SSR com Frameworks de Frontend

Frameworks modernos como React, Vue.js e Next.js possuem suporte para SSR.

Exemplo Prático: Implementação de SSR com Node.js e Express

1. Estrutura do Projeto

```

├── src/
│   ├── server.js      # Arquivo principal do servidor Express
│   ├── db.js          # Conexão com o MySQL
│   │
│   ├── routes/        # Rotas do sistema
│   │   ├── pages.js   # Rotas SSR (Home, About, etc.)
│   │   └── usuarios.js # Rotas do CRUD de usuários
│   │
│   ├── views/         # Templates (EJS)
│   │   ├── layouts/   # Layout principal (base da página)
│   │   ├── partials/  # Header e Footer reutilizáveis
│   │   └── *.ejs      # Páginas renderizadas
│   │
│   ├── public/
│   │   └── css/        # Arquivos estáticos (estilo)
│   │
│   ├── sql/           # Scripts de banco de dados
│   │
│   ├── .env           # Variáveis de ambiente
│   ├── docker-compose.yml # MySQL via Docker
│   └── package.json    # Dependências e scripts

```

Uma pequena pausa

Ao longo deste material, serão apresentados conceitos e exemplos práticos antes da execução dos testes locais.

Essa abordagem permite compreender a função de cada componente do projeto e como suas responsabilidades se relacionam com as ações executadas pelo sistema.

5.2: Introdução às Templating Engines (EJS, Pug)

O que são Templating Engines?

- Ferramentas que permitem criar páginas HTML dinâmicas no lado do servidor.
- Permitem a injeção de dados em tempo real e organização de layouts reutilizáveis.

EJS (Embedded JavaScript):

1. Sintaxe similar ao HTML.
2. Tags de uso: `<%= %>` para saída de variáveis e `<% %>` para código JavaScript.

Pug:

- Sintaxe minimalista baseada em indentação.
- Focado em simplicidade e organização.

Exemplo de Comparativo:

- EJS:

```
<h1> Olá, <%= nome %>! </h1>
```

- Pug:

```
h1 Olá, #{nome}!
```

5.3: Configuração de templates dinâmicos com EJS

Esse trecho configura o Express para utilizar o EJS como motor de templates, permitindo gerar páginas HTML dinâmicas no servidor. Arquivo **server.js**

```
// Views / templating (EJS)
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));
app.use(layouts);
app.set('layout', 'layouts/main');
```

```
app.set('view engine', 'ejs');
```

Define o EJS como mecanismo de renderização de páginas. Assim, o Express entende que os arquivos `.ejs` serão usados como templates.

```
app.set('views', path.join(__dirname, 'views'));
```

Define a pasta onde ficam os arquivos de template. Nesse caso, todos os arquivos `.ejs` ficam dentro da pasta `views`.

```
app.use(layouts);
```

Ativa o uso de layouts reutilizáveis. Isso permite criar uma estrutura padrão de página (ex: header, footer, menu) sem repetir código.

```
app.set('layout', 'layouts/main');
```

Define o layout principal da aplicação. Todas as páginas renderizadas usarão esse layout como base.

Essas configurações dizem ao servidor:

“Use EJS para montar páginas HTML, busque os arquivos na pasta `views` e aplique um layout padrão para todas as páginas.”

Criando um Template Simples:

1. Crie a pasta `views`.
2. Adicione o arquivo `index.ejs`:

```
<div class="card">
  <h1><%= message %></h1>
  <p>Esta página foi gerada no <strong>servidor</strong> e enviada como HTML pronto para o navegador.</p>
  <p>Use o menu para navegar entre exemplos de templates, partials, MySQL e formulários.</p>
</div>
```

Renderizando o Template: `routes/pages.js`

```
// Home (SSR)
router.get('/', (req, res) => {
  res.render('index', {
    title: 'SSR com Node.js',
    message: 'Bem-vindo ao SSR! (renderizado no servidor)',
  });
});
```

5.4: Renderizando dados do MySQL em páginas dinâmicas

Conexão ao Banco e Envio de Dados: `routes/usuarios.js`

```
// Listar usuários (SSR)
router.get('/', async (req, res) => {
  try {
    const [usuarios] = await pool.query('SELECT * FROM usuarios ORDER BY id DESC');
    res.render('usuarios', { title: 'Usuários', usuarios });
  } catch (err) {
    res.status(500).render('usuarios', { title: 'Usuários', usuarios: [], error: err.message });
  }
});
```

Template Exemplo (`usuarios.ejs`):

```
<div class="card">
  <h1>Lista de Usuários</h1>

  <% if (typeof error !== 'undefined') { %>
    <div class="error"><strong>Erro:</strong> <%= error %></div>
  <% } %>

  <table>
    <thead>
      <tr><th>ID</th><th>Nome</th><th>Email</th><th>Criado em</th></tr>
    </thead>
    <tbody>
      <% usuarios.forEach(u => { %>
        <tr>
          <td><%= u.id %></td>
          <td><%= u.nome %></td>
          <td><%= u.email %></td>
          <td><%= u.created_at %></td>
        </tr>
      <% }) %>
    </tbody>
  </table>
</div>
```

5.5: Trabalhando com partials e layouts reutilizáveis

O que são Partials?

- Arquivos reutilizáveis que contêm partes de um layout comum, como cabeçalhos ou rodapés.

Criando um Partial (`header.ejs`):

```
<header>
  <strong>UniSenai</strong> - Módulo 5 (SSR / Templating)

  <nav style="margin-top:10px">
    <a href="/" style="color:■ #fff;margin-right:10px">Home</a>
    <a href="/about" style="color:■ #fff;margin-right:10px">About</a>
    <a href="/usuarios" style="color:■ #fff;margin-right:10px">Usuários</a>
    <a href="/usuarios/novo" style="color:■ #fff;margin-right:10px">Cadastrar</a>
    <a href="/ex-api" style="color:■ #fff">API (mock)</a>
  </nav>
</header>
```

Incluindo Partials no Template:

```
<!doctype html>
<html lang="pt-br">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title><%= title %></title>
  <link rel="stylesheet" href="/static/css/style.css" />
</head>
<body>
  <%- include('../partials/header') %>
  <main>
    <%- body %>
  </main>
  <%- include('../partials/footer') %>
</body>
</html>
```

5.6: Formulários no Express: Validação e envio de dados para o MySQL

Configuração para Processar Formulários:

```
// Body parsing (substitui body-parser)
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
```

Criação de Formulário:


```
<form action="/usuarios" method="POST">
  <p>
    <label>Nome</label><br/>
    <input type="text" name="nome" placeholder="Nome" required />
  </p>
  <p>
    <label>Email</label><br/>
    <input type="email" name="email" placeholder="Email" required />
  </p>
  <button type="submit">Enviar</button>
</form>
```

Rota para Receber Dados:

```
// Receber form (POST)
router.post('/', async (req, res) => {
  const { nome, email } = req.body || {};
  const errors = [];

  if(!nome || String(nome).trim().length < 2) errors.push('Nome é obrigatório (mín. 2 caracteres).');
  if(!email || !String(email).includes('@')) errors.push('Email inválido.');
```

```
  if(errors.length) {
    return res.status(400).render('usuario-form', {
      title: 'Cadastrar usuário',
      values: { nome, email },
      errors
    });
  }

  try {
    await pool.execute('INSERT INTO usuarios (nome, email) VALUES (?, ?)', [nome.trim(), email.trim()]);
    return res.redirect('/usuarios');
  } catch (err) {
    // Ex.: email duplicado
    return res.status(500).render('usuario-form', {
      title: 'Cadastrar usuário',
      values: { nome, email },
      errors: [err.message]
    });
  }
});
```

 **Observação:** Consulte aqui o material de exemplo com práticas para treinar suas habilidades: [Módulo 5](#)

Atividades Propostas

Teóricas:

- Explique as diferenças entre Server-Side Rendering (SSR) e Client-Side Rendering (CSR).
- Liste três vantagens e três desvantagens do SSR.
- Qual é o impacto do SSR no SEO? Justifique com exemplos.

- Explique a diferença entre hidratação e renderização inicial no SSR.
- Pesquise e explique como o Next.js utiliza SSR para melhorar a experiência do desenvolvedor.

Práticas:

1. Adicione uma nova rota `/about` que também use SSR para renderizar um template com informações fictícias sobre o site.
2. Utilize a biblioteca Handlebars como template engine em vez de EJS.
3. Integre uma API externa (ex.: JSONPlaceholder) e exiba os dados obtidos no template.

Lista de Exercícios

Questões Teóricas

1. O que são templating engines e qual sua utilidade?
2. Explique as diferenças entre EJS e Pug.
3. Como configurar o EJS em um projeto Express?
4. O que são partials e como eles auxiliam na reutilização de código?
5. Como os dados do MySQL podem ser renderizados em um template?
6. Explique o papel do `body-parser` em aplicações Express.
7. Quais são as boas práticas ao organizar templates em um projeto?
8. Como funciona a inclusão de placeholders em layouts do EJS?
9. Liste três aplicações práticas para o uso de formulários no Express.
10. Por que é importante validar dados recebidos de formulários?

Questões Práticas

- Configure o EJS em um projeto Express e crie um template básico.
- Implemente um template que liste usuários cadastrados em um banco MySQL.
- Crie um layout com cabeçalho e rodapé utilizando partials.
- Desenvolva um formulário para cadastrar novos usuários no banco de dados.
- Configure uma rota POST que receba os dados do formulário e insira no MySQL.
- Crie um template que exiba mensagens de erro ao processar dados.
- Implemente um sistema de navegação entre páginas utilizando layouts reutilizáveis.
- Crie um placeholder dinâmico para o título da página em um layout.
- Integre dados do MySQL em um menu dinâmico renderizado via EJS.
- Valide os campos de entrada do formulário e exiba mensagens de erro caso os dados sejam inválidos.